

Edge cases (context limit) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Simple Version

```
from openai import OpenAI

client = OpenAI()

huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except Exception as e:
    print("Error type:", type(e).__name__)
    print("Message:", str(e)[:200])
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version — 2 approaches

Approach 1 — catch the specific error type

```
import openai
from openai import OpenAI

client = OpenAI()

huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    print(f"Error type: {type(e).__name__}")
    print(f"Message: {str(e)[:200]}")
```

Approach 2 — check the size before sending, using `tiktoken`

Instead of finding out only after the API rejects the call, count tokens yourself first and decide what to do (trim, summarize, or refuse) before ever making the request.

```
import tiktoken
from openai import OpenAI

client = OpenAI()

MAX_TOKENS = 120_000 # leave headroom below the model's real limit

def count_tokens(text: str, model: str = "gpt-4o-mini") -> int:
    encoding = tiktoken.encoding_for_model(model)
    return len(encoding.encode(text))

huge_input = "word " * 200_000
token_count = count_tokens(huge_input)
```

```
if token_count > MAX_TOKENS:
    print(f"Input is {token_count} tokens – too long, refusing to send.")
else:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
```

Why Approach 2 is what you'd actually ship: Approach 1 only tells you about the problem *after* wasting a network round-trip on a request that was always going to fail. Approach 2 catches it locally, instantly, for free — and gives you a chance to react intelligently (trim the oldest messages, summarize, or tell the user clearly) instead of just showing an error. Real chat apps almost always do both: check locally as the first line of defense, and still keep the `try/except` from Approach 1 as a safety net for anything the local count got wrong.